

04. Ensemble Learning

Algorithms implemented: Random Forest Regression (RFR), Random Forest Classification (RFC), XGBoost, AdaBoost, CatBoost.

Datasets: California Housing (regression) and Heart Disease (`data/heart_disease.csv` , classification).

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor,
AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

# ---- Load the California Housing dataset (real-world regression benchmark) ----
housing = fetch_california_housing(as_frame=True)
# Sample 5,000 rows so every ensemble trains quickly while remaining representative
df_reg = housing.frame.sample(n=5000, random_state=42)

X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
y_reg = df_reg['MedHouseVal'] # target: median house value

# ---- Train/test split (80% / 20%) ----
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.20, random_state=42
)

print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples: {X_test_reg.shape[0]}")
df_reg.head()
```

Regression Training Samples: 4000, Test Samples: 1000

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
20046	1.6812	25.0	4.192201	1.022284	1392.0	3.877437	36.06	-119.01	0.47700
3024	2.5313	30.0	5.039384	1.193493	1565.0	2.679795	35.14	-119.46	0.45800
15663	3.4801	52.0	3.977155	1.185877	1310.0	1.360332	37.80	-122.44	5.00001
20484	5.7376	17.0	6.163636	1.020202	1705.0	3.444444	34.28	-118.72	2.18600
9814	3.7250	34.0	5.492991	1.028037	1063.0	2.483645	36.62	-121.93	2.78000

1. Random Forest Regression (RFR)

Step-by-Step Algorithm (Random Forest Regression)

1. Draw B bootstrap samples from the training data (sampling with replacement).
2. Grow one decision tree per bootstrap sample; at each split consider only a random subset of features.
3. Let every tree predict independently on the test point.
4. Average the B tree predictions: $\hat{f}_{RF}(x) = (1/B) \sum T_b(x)$.
5. Use out-of-bag samples (not drawn in a bootstrap) to estimate generalization (OOB score).

```
# ---- Random Forest Regression (bagging of decision trees) ----
# n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag samples
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True, random_state=42, n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_rf = rf_reg.predict(X_test_reg)
r2_rf = r2_score(y_test_reg, y_pred_rf)
rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf)
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)

# explained variance (1.0 = perfect)
# error in target units
# robust average error

print("Random Forest Regressor Results:")
print(f" - Out-of-Bag (OOB) R2 Score: {rf_reg.oob_score_:.4f}")
print(f" - Test R2 Score: {r2_rf:.4f}")
print(f" - Test RMSE: {rmse_rf:.4f}")
print(f" - Test MAE: {mae_rf:.4f}")
```

```
Random Forest Regressor Results:
- Out-of-Bag (OOB) R2 Score: 0.7513
- Test R2 Score: 0.7415
- Test RMSE: 0.5908
- Test MAE: 0.3999
```

2. XGBoost Regression

Step-by-Step Algorithm (XGBoost Regression)

1. Start with a constant prediction (e.g. the mean target).
2. For each boosting round t : a. Compute first-order gradients g_i and second-order Hessians h_i of the loss. b. Grow a new tree that minimizes the regularized objective: $\sum [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$. c. $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$ penalizes tree complexity (T leaves) and leaf weights.
3. Add the new tree to the ensemble with a shrinkage factor (learning_rate).
4. Repeat steps 2–3 for $n_estimators$ rounds; prediction = sum of all trees.

```
# ---- XGBoost Regression (gradient boosting with 2nd-order optimization) ----
# subsample/colsample add stochasticity; reg_alpha/reg_lambda are L1/L2 regularizers.
xgb_reg = xgb.XGBRegressor(
    n_estimators=150,
    learning_rate=0.08,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42
)
xgb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_xgb = xgb_reg.predict(X_test_reg)
r2_xgb = r2_score(y_test_reg, y_pred_xgb)
rmse_xgb = root_mean_squared_error(y_test_reg, y_pred_xgb)
mae_xgb = mean_absolute_error(y_test_reg, y_pred_xgb)

print("XGBoost Regressor Results:")
print(f" - Test R2 Score: {r2_xgb:.4f}")
print(f" - Test RMSE: {rmse_xgb:.4f}")
print(f" - Test MAE: {mae_xgb:.4f}")
```

```
XGBoost Regressor Results:
- Test R2 Score: 0.8025
- Test RMSE: 0.5165
- Test MAE: 0.3467
```

3. AdaBoost Regression

Step-by-Step Algorithm (AdaBoost Regression)

1. Initialize equal weights w_i for all training samples.
2. Train a weak regressor on the weighted data.
3. Compute the relative error $e_i = |y_i - \hat{y}_m(x_i)| / \max_j |y_j - \hat{y}_m(x_j)|$.

4. Set the learner weight $\beta_m = \bar{L}_m / (1 - \bar{L}_m)$ from the average weighted loss.
5. Increase weights of poorly predicted samples: $w_i \leftarrow w_i \cdot \beta_m^{(1 - e_i)}$.
6. Repeat steps 2–5 for M rounds, then take the weighted median of all predictions.

```
# ---- AdaBoost Regression (sequential re-weighting of weak regressors) ----
# Each new weak learner focuses more on the samples the previous ones predicted poorly.
ada_reg = AdaBoostRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
ada_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_ada = ada_reg.predict(X_test_reg)
r2_ada = r2_score(y_test_reg, y_pred_ada)
rmse_ada = root_mean_squared_error(y_test_reg, y_pred_ada)
mae_ada = mean_absolute_error(y_test_reg, y_pred_ada)

print(f"AdaBoost Regressor Results:")
print(f" - Test R² Score: {r2_ada:.4f}")
print(f" - Test RMSE:      {rmse_ada:.4f}")
print(f" - Test MAE:        {mae_ada:.4f}")
```

```
AdaBoost Regressor Results:
- Test R² Score: 0.5888
- Test RMSE:    0.7452
- Test MAE:     0.5842
```

4. CatBoost Regression

Step-by-Step Algorithm (CatBoost Regression)

1. Build **oblivious (symmetric) trees**: the same split condition is used at every node of a depth level.
2. Use **ordered boosting**: train each tree on a random permutation of the data to avoid target leakage/prediction shift.
3. Compute leaf values from the ordered statistics, then add the tree to the ensemble with the learning rate.
4. Repeat for the configured number of iterations.
5. Predict by summing the values of all trees for the input.

```
# ---- CatBoost Regression (ordered boosting + oblivious trees) ----
# l2_leaf_reg penalizes large leaf values; verbose=False keeps the notebook output clean.
cb_reg = cb.CatBoostRegressor(
    iterations=200,
    learning_rate=0.08,
    depth=6,
    l2_leaf_reg=3.0,
    random_seed=42,
    verbose=False
)
cb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_cb = cb_reg.predict(X_test_reg)
r2_cb = r2_score(y_test_reg, y_pred_cb)
rmse_cb = root_mean_squared_error(y_test_reg, y_pred_cb)
mae_cb = mean_absolute_error(y_test_reg, y_pred_cb)

print(f"CatBoost Regressor Results:")
print(f" - Test R² Score: {r2_cb:.4f}")
print(f" - Test RMSE:      {rmse_cb:.4f}")
print(f" - Test MAE:        {mae_cb:.4f}")
```

```
CatBoost Regressor Results:
- Test R² Score: 0.8029
- Test RMSE:    0.5160
- Test MAE:     0.3535
```

Regression Summary

```
# ---- Side-by-side regression comparison ----
reg_comparison = pd.DataFrame([
    {"Model": "Random Forest Regressor (RFR)", "R² Score": r2_rf, "RMSE": rmse_rf, "MAE": mae_rf},
    {"Model": "AdaBoost Regressor", "R² Score": r2_ada, "RMSE": rmse_ada, "MAE": mae_ada},
    {"Model": "XGBoost Regressor", "R² Score": r2_xgb, "RMSE": rmse_xgb, "MAE": mae_xgb},
    {"Model": "CatBoost Regressor", "R² Score": r2_cb, "RMSE": rmse_cb, "MAE": mae_cb}
]).set_index("Model")
```

```
print("Ensemble Regression Comparison on California Housing:")
# Green = best value (max R², min RMSE/MAE)
display(reg_comparison.style.highlight_max(subset=['R² Score'], color='lightgreen')
        .highlight_min(subset=['RMSE', 'MAE'], color='lightgreen'))
```

Ensemble Regression Comparison on California Housing:

	R² Score	RMSE	MAE
Model			
Random Forest Regressor (RFR)	0.741531	0.590846	0.399898
AdaBoost Regressor	0.588827	0.745216	0.584228
XGBoost Regressor	0.802459	0.516534	0.346691
CatBoost Regressor	0.802873	0.515992	0.353549

5. Random Forest Classification (RFC)

Step-by-Step Algorithm (Random Forest Classification)

1. Draw B bootstrap samples and grow a tree per sample (random feature subset at each split).
2. Each tree casts a unit vote for the class at input x.
3. Final prediction = majority vote: $\hat{C}_{RF}(x) = \arg\max_k \sum_b I(T_b(x) = k)$.
4. Class probabilities = fraction of trees voting for each class (used for ROC-AUC).

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Heart Disease dataset: 303 rows x 14 columns

Classification Training Set: (227, 13), Test Set: (76, 13)

Random Forest Classifier: Accuracy = 0.7763, F1 = 0.8046, ROC-AUC = 0.8704

6. XGBoost Classification

Step-by-Step Algorithm (XGBoost Classification)

1. Use the gradient-boosted tree procedure with the logistic loss on the binary heart-disease target.
2. Each round fits a tree to the 1st/2nd-order gradients of the logistic loss.
3. Regularization (L1/L2, depth limits, subsampling) controls overfitting.

4. Convert the final margin to probabilities with the sigmoid for ROC-AUC.

```
# ---- XGBoost classification on the same stratified split ----
xgb_clf = xgb.XGBClassifier(
    n_estimators=120, learning_rate=0.08, max_depth=4,
    subsample=0.8, colsample_bytree=0.8,
    random_state=42, eval_metric='logloss'
)
xgb_clf.fit(X_train_clf, y_train_clf)

y_pred_xgb_c = xgb_clf.predict(X_test_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]

acc_xgb_c = accuracy_score(y_test_clf, y_pred_xgb_c)
f1_xgb_c = f1_score(y_test_clf, y_pred_xgb_c)
auc_xgb_c = roc_auc_score(y_test_clf, y_prob_xgb_c)
print(f"XGBoost Classifier: Accuracy = {acc_xgb_c:.4f}, F1 = {f1_xgb_c:.4f}, ROC-AUC = {auc_xgb_c:.4f}")

XGBoost Classifier: Accuracy = 0.7763, F1 = 0.8132, ROC-AUC = 0.8509
```

7. AdaBoost Classification

Step-by-Step Algorithm (AdaBoost Classification)

1. Initialize equal sample weights.
2. Train a weak learner (depth-1 stump) on the weighted samples.
3. Compute its weighted error and learner weight $\alpha = 1/2 \ln((1-\epsilon)/\epsilon)$.
4. Increase weights of misclassified samples and repeat.
5. Final prediction = weighted vote of all stumps.

```
# ---- AdaBoost classification ----
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)

y_pred_ada_c = ada_clf.predict(X_test_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[:, 1]

acc_ada_c = accuracy_score(y_test_clf, y_pred_ada_c)
f1_ada_c = f1_score(y_test_clf, y_pred_ada_c)
auc_ada_c = roc_auc_score(y_test_clf, y_prob_ada_c)
print(f"AdaBoost Classifier: Accuracy = {acc_ada_c:.4f}, F1 = {f1_ada_c:.4f}, ROC-AUC = {auc_ada_c:.4f}")

AdaBoost Classifier: Accuracy = 0.7895, F1 = 0.8182, ROC-AUC = 0.8714
```

8. CatBoost Classification (Native Categorical Handling)

Step-by-Step Algorithm (CatBoost Classification)

1. Identify categorical columns and pass them through `cat_features`.
2. Encode categories internally with ordered target statistics (computed on random permutations).
3. Build oblivious trees with ordered boosting.
4. Output class probabilities via the sigmoid of the summed tree values.

```
# ---- CatBoost classification with native categorical features ----
# Identify categorical columns in the raw (unencoded) heart data
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
X_raw_cb = X_clf_raw.copy()
for col in categorical_cols:
    X_raw_cb[col] = X_raw_cb[col].astype(str)

# Same stratified split on the raw feature table
X_tr_cb, X_te_cb, y_tr_cb, y_te_cb = train_test_split(
    X_raw_cb, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

cb_clf = cb.CatBoostClassifier(
    iterations=150, learning_rate=0.08, depth=5,
    cat_features=categorical_cols, random_seed=42, verbose=False
)
cb_clf.fit(X_tr_cb, y_tr_cb)
```

```
y_pred_cb_c = cb_clf.predict(X_te_cb)
y_prob_cb_c = cb_clf.predict_proba(X_te_cb)[: , 1]

acc_cb_c = accuracy_score(y_te_cb, y_pred_cb_c)
f1_cb_c = f1_score(y_te_cb, y_pred_cb_c)
auc_cb_c = roc_auc_score(y_te_cb, y_prob_cb_c)
print(f"CatBoost Classifier: Accuracy = {acc_cb_c:.4f}, F1 = {f1_cb_c:.4f}, ROC-AUC = {auc_cb_c:.4f}")
```

CatBoost Classifier: Accuracy = 0.7632, F1 = 0.7955, ROC-AUC = 0.8551

9. Classification Summary

```
# ---- Classification comparison: RFC vs XGBoost vs AdaBoost vs CatBoost ----
clf_comparison = pd.DataFrame([
    {"Model": "Random Forest Classifier (RFC)", "Accuracy": acc_rfc, "F1-Score": f1_rfc, "ROC-AUC": auc_rfc},
    {"Model": "XGBoost Classifier", "Accuracy": acc_xgb_c, "F1-Score": f1_xgb_c, "ROC-AUC":
auc_xgb_c},
    {"Model": "AdaBoost Classifier", "Accuracy": acc_ada_c, "F1-Score": f1_ada_c, "ROC-AUC":
auc_ada_c},
    {"Model": "CatBoost Classifier", "Accuracy": acc_cb_c, "F1-Score": f1_cb_c, "ROC-AUC": auc_cb_c},
]).set_index("Model")

print("Ensemble Classification Comparison on Heart Disease:")
display(clf_comparison.style.highlight_max(subset=['Accuracy', 'F1-Score', 'ROC-AUC'], color='lightgreen'))
```

Ensemble Classification Comparison on Heart Disease:

	Accuracy	F1-Score	ROC-AUC
Model			
Random Forest Classifier (RFC)	0.776316	0.804598	0.870383
XGBoost Classifier	0.776316	0.813187	0.850871
AdaBoost Classifier	0.789474	0.818182	0.871429
CatBoost Classifier	0.763158	0.795455	0.855052